□ LLD HandNotes — Class vs Object vs Variable vs Method vs Relationship

(Zero se Confident — Interview Ready)

> ***Mentor ki baat:*** *Yeh notes bookish nahi hai. Jaise tum class me bunk karke kisi senior se ek-ek line samajh rahe ho, waise hi likha hai. Ek sentence se pura design nikalenge — step by step, arrow by arrow.*

1□ TOPIC NAME

"Librarian, Member ko Book issue karta hai, Library ke through." (Class vs Variable vs Method vs Relationship — full breakdown)

2□ PROBLEM / CONCEPT INTRODUCTION

Sentence hai:

> ***"Library ka Librarian, Member ko Book issue karta hai."***

Yeh ek normal Hindi/Hinglish sentence hai — lekin agar tum ise **LLD ki nazar (eyes)** se padho, toh isme already poora system design chhupa hai.

Mentor tip: Jab bhi koi problem statement mile, usko ek normal kahani na samjho — usko **"nouns + verbs + values + relationships"** ka mixture samjho. Yeh skill hi LLD interview clear karwati hai.

3□ NOUN EXTRACTION (Pehchaan: kaun-kaun "cheez/entity" hai)

Noun	Kya lagta hai?
Library	Entity — ek jagah/system jisme sab kuch hota hai
Librarian	Entity — ek insaan jo kaam karta hai
Member	Entity — ek insaan jo book leta hai
Book	Entity — jo cheez issue ho rahi hai

➡ **Arrow Logic:** `noun → possible class`

> *Yaad rakho:* ***"har noun class nahi hota"*** *— yeh hum Section 9 me cover karenge.*

4□ VERB EXTRACTION (Pehchaan: kaun-kaun "action/kaam" ho raha hai)

Verb	Action kiska hai?
issue karta hai	Librarian → Book → Member
(implied) borrow karta hai	Member ka action
(implied) return karega	Member ka future action
(implied) manage karta hai	Librarian ka role

➡ **Arrow Logic:** `verb → possible method`

5□ VARIABLE CANDIDATES (Sirf "value" jo store hoti hai, action nahi karti)

Value	Kis class ke andar jayega
`bookTitle`, `bookId`, `author`	Book class ke andar
`memberName`, `memberId`	Member class ke andar
`librarianName`, `employeeId`	Librarian class ke andar
`issueDate`, `dueDate`	IssueRecord (advanced) ya Book ke andar (simple)
`fineAmount`	Simple app me variable, Advanced app me alag class

➡ **Arrow Logic:** `simple fixed value → variable` (no behavior, no identity of its own)

6□ CLASS CANDIDATES (Final shortlist)

Class	Kyun class?
`Library`	Pura system control karti hai — controller jaisa role
`Librarian`	Apna data (naam, id) + apna kaam (issue karna) — identity + behavior

Class	Kyun class?
`Member`	Apna data + apna kaam (borrow karna) — identity + behavior
`Book`	Apna data (title, author) + status (available/issued) — identity + behavior
`IssueRecord` *(advanced only)*	Jab issueDate, dueDate, fine sab serious tracking chahiye

---

7□ RELATIONSHIP IDENTIFICATION

```
Library  ──HAS-A (composition)──▶  Book[]
Library  ──HAS-A (aggregation)──▶  Librarian[]
Librarian──ASSOCIATION (issues)──▶  Book
Librarian──ASSOCIATION (gives to)──▶  Member
Member   ──ASSOCIATION (borrows)──▶ Book
Librarian, Member ──IS-A──▶ Person (advanced, agar Person class banaye)
```

Relationship	Type	Reason
Library — Book	**Composition**	Book ka existence Library ke bina meaningless hai (in this system)
Library — Librarian	**Aggregation**	Librarian, Library band hone ke baad bhi insaan ke roop me exist karta hai
Librarian — Book (issue action)	**Association**	Temporary use, koi ownership nahi
Member — Book (borrow)	**Association**	Member kuch time ke liye use karta hai, owner nahi
Librarian / Member — Person	**Inheritance (IS-A)**	Dono "Person" hi hai, bas role alag

➡ Arrow Logic Recap:

```
strong ownership   → composition   (part dies with whole)
weak ownership     → aggregation   (part survives without whole)
loose/temporary use→ association
"is a" relationship→ inheritance
```

8□ CONTROLLER CLASS

`Library` is the controller in simple design.

- It manages list of books, list of members, list of librarians.
- It exposes methods like `issueBook()`, `returnBook()`.

Advanced design me: `LibraryService` ya `IssueManager` separate controller class banta hai, taaki `Library` sirf data hold kare aur business logic alag rahe (Single Responsibility Principle).

9□ FINAL DESIGN DECISION

□ Small Project View

```
Library
 ├── List<Book> books
 ├── List<Member> members
 └── issueBook(bookId, memberId)
```

Simple, sab kuch ek hi jagah handle.

□ Large/Scalable Project View

```
Library --- LibraryService --- IssueRecord --- FineCalculator
                            --- NotificationService
```

Har responsibility alag class — taaki system grow kar sake.

Mentor ki baat: Interviewer dekhna chahta hai ki tumhe pata hai system chhota ho toh kya, bada ho toh kya badlega. Yehi maturity dikhati hai.

□ PYTHON CODE

```
class Book:
    def __init__(self, book_id, title, author):
        self.book_id = book_id          # variable: identity
        self.title = title              # variable: simple data
        self.author = author            # variable: simple data
        self.is_available = True        # variable: status flag

class Member:
    def __init__(self, member_id, name):
        self.member_id = member_id
        self.name = name

class Librarian:
    def __init__(self, employee_id, name):
        self.employee_id = employee_id
        self.name = name

    def issue_book(self, book: Book, member: Member):
        if book.is_available:
            book.is_available = False
            print(f"{self.name} issued '{book.title}' to {member.name}")
        else:
            print(f"'{book.title}' is already issued.")

class Library:
    def __init__(self):
        self.books = []         # composition: books belong to library
        self.members = []       # aggregation: members exist independently
        self.librarian = None

    def add_book(self, book: Book):
        self.books.append(book)
```

1□1□ JAVA CODE

```
class Book {
    String bookId;
    String title;
    String author;
    boolean isAvailable = true;   // variable: status flag

    Book(String bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }
}

class Member {
    String memberId;
    String name;

    Member(String memberId, String name) {
        this.memberId = memberId;
        this.name = name;
    }
}

class Librarian {
    String employeeId;
    String name;

    Librarian(String employeeId, String name) {
        this.employeeId = employeeId;
        this.name = name;
    }

    void issueBook(Book book, Member member) {
        if (book.isAvailable) {
            book.isAvailable = false;
            System.out.println(this.name + " issued " + book.title + " to " + member.name);
        } else {
            System.out.println(book.title + " is already issued.");
        }
    }
}

class Library {
    List<Book> books = new ArrayList<>();     // composition
    List<Member> members = new ArrayList<>(); // aggregation
```

```
    Librarian librarian;

    void addBook(Book book) {
        books.add(book);
    }
}
```

1□2□ LINE-BY-LINE EXPLANATION

- `book_id, title, author` → ye **variables** hai kyunki yeh sirf data store karte hai, koi action nahi karte.
- `is_available` → ye bhi variable hai, lekin yeh **state/status** represent karta hai — issue/return logic isi pe depend karega.
- `issue_book()` method **Librarian** class ke andar hai, kyunki real life me **action karne wala Librarian hai**, Book khud issue nahi hoti.
- `Library` class ke andar `books` list hai — yeh **composition** dikhata hai (Library object delete hua toh us list ke books bhi gaye, in this design).
- Python me `self` explicit likhna padta hai, Java me **constructor +** `this` keyword use hota hai — concept same hai, syntax alag.
- Python **dynamically typed** hai (type nahi likhna padta), Java **statically typed** hai (`String`, `boolean` likhna zaruri hai).
- Dono languages me: **class = blueprint, object = real instance, method = behavior, variable = data** — yeh concept universal hai.

1□3□ MCQs WITH ANSWERS

Q1. `bookTitle` **ko class banayenge ya variable?**

a. Class
b. Variable
c. Method
d. Controller

□ **Answer: b) Variable** — Reason: Yeh sirf ek text value hai, uska apna behavior nahi hai.

Q2. `Library` **aur** `Book` **ke beech relationship?**

a. Inheritance
b. Composition
c. Association
d. None

□ **Answer: b) Composition** — Reason: Book, Library ke bina is system me exist nahi karti (strong ownership).

Q3. `issueBook()` **method kis class me hona chahiye?**

a. Book

b. Member

c. Librarian

d. Anywhere

 Answer: c) Librarian — Reason: Real world me action perform karne wala Librarian hai, isliye behavior usi class me.

Q4. Agar Member, Library band hone ke baad bhi exist karta hai, toh relationship kya?

a. Composition

b. Aggregation

c. Inheritance

d. None

 Answer: b) Aggregation — Reason: Part (Member) whole (Library) ke bina independently exist kar sakta hai.

Q5. Librarian aur Member dono "Person" hai — yeh kis concept ka example hai?

a. Composition

b. Aggregation

c. Inheritance (IS-A)

d. Association

 Answer: c) Inheritance (IS-A) — Reason: "Librarian IS-A Person", "Member IS-A Person" — dono ek common parent share karte hai.

1□4□ EDGE CASES / TRICKY SCENARIOS

Scenario	Small Project	Large Project
Rating (book ki rating)	`float rating` — simple variable	`Rating` class (with reviewer, comment, timestamp)
Book	Class	Splits into `Book` + `BookCopy` (jab multiple physical copies track karni ho)
Fine	`fineAmount` variable in Member	`FineCalculator` class with rules engine
Issue action	Method `issueBook()` in Librarian	Separate `IssueRecord` class + `IssueService` controller
Address	`String address` variable	`Address` class (street, city, pincode) jab validation/formatting complex ho

Mentor ki baat: Yehi flexibility samajhna LLD ka asli core hai — "design depends on scale and requirement, not on a fixed rule."

1□5□ INTERVIEW NOTES

Q: Why class and not variable?

> *"Kyunki is entity ka apna identity aur behavior hai — sirf data nahi, action bhi karta hai."*

Q: Why method and not separate class?

> *"Abhi system ki complexity itni nahi hai ki yeh action apna alag state maintain kare, isliye method hi sufficient hai."*

Q: Why inheritance and not composition?

> *"Kyunki relationship 'IS-A' type hai, na ki 'HAS-A' — Librarian khud ek Person hai, Person ko hold nahi karta."*

Q: Agar dono options possible hai (class ya variable)?

> *"Main bolunga — 'Currently requirement simple hai isliye variable rakha, lekin agar future me isko apna behavior/validation chahiye toh main isse class me convert kar dunga.' Yeh batata hai ki tum scalability soch rahe ho."*

Common mistakes:

- Har noun ko class bana dena (over-engineering).
- Composition aur Aggregation ko same samajh lena.
- Controller class ko bhul jaana — flow kaun chalata hai, woh batana zaruri hai.
- Method aur Class ka confusion — "Issue" verb hai, action hai, class mat bana do jab tak complexity na ho.

1□6□ QUICK REVISION SUMMARY

5-line revision:

1. Noun → Class (mostly), Verb → Method, Simple value → Variable.
2. Strong ownership = Composition, Weak ownership = Aggregation, Loose use = Association, IS-A = Inheritance.
3. Controller class = jo flow ko manage kare (yahan `Library`).
4. Design depends on project scale — same entity class ya variable, depends on requirement.
5. Python aur Java me syntax alag, concept same.

5 takeaways:

- Identity + Behavior wala noun = Class.

- Sirf data wali cheez = Variable.
- Action karne wala entity hi method ka owner hota hai.
- Relationship decide karne se pehle "ownership strength" socho.
- Interview me apna design justify karna sabse important skill hai.

5 traps to avoid:

- Har cheez ko class bana dena.
- Composition/Aggregation confuse karna.
- Controller class identify na karna.
- Verb ko hamesha method maan lena.
- Scalability ka angle bhool jaana.

5 practice questions:

1. "Customer, restaurant se food order karta hai, delivery boy deliver karta hai." — Classes/relationships nikaalo.
2. "ATM machine, card verify karke cash deliver karti hai." — Controller class kaun hai?
3. "Parking lot me car ko slot assign hota hai." — Composition ya Aggregation?
4. "Teacher, student ko marks deta hai." — IS-A relationship kaha hai?
5. "WhatsApp user, message bhejta hai group me." — Class candidates list karo.

Mini assignment:

> *Apna ek real-life sentence likho (jaise "Doctor patient ko appointment deta hai") aur is poori sheet ke 16 steps khud follow karke design bana ke dekho. Jab tum khud kar paoge, tab samajhna ki concept clear hai.*

□ SAME LOGIC, DIFFERENT DOMAINS (Quick Comparison)

Domain	Controller Class	Composition Example	Aggregation Example	Inheritance Example
ATM	`ATM`	ATM — CashDispenser	ATM — BankAccount	DebitCard, CreditCard → Card
School	`School`	School — Classroom	School — Teacher	Teacher, Student → Person
Food Delivery	`OrderService`	Order — OrderItem	Order — DeliveryBoy	Customer, Restaurant → User
Parking Lot	`ParkingLot`	ParkingLot — ParkingSlot	ParkingLot — Vehicle	Car, Bike → Vehicle
Movie Booking	`BookingService`	Booking — Seat	Booking — Customer	Member, Guest → User
WhatsApp	`ChatService`	Group — Message	Group — User	IndividualChat, GroupChat → Chat
Chess	`Game`	Board — Cell	Game — Player	King, Queen → Piece

➡ **Pattern hamesha same rehta hai** — sirf naam aur domain badalta hai. Isi pattern ko pakad lo, koi bhi LLD problem solve ho jayegi.